

Best Bike Paths - Hu Peng, Zhang Zihao



POLITECNICO
MILANO 1863

Implementation and Test deliverable

Deliverable:	ITD
Title:	Implementation and Test deliverable
Authors:	Hu Peng, Zhang Zihao
Version:	1.0
Date:	31-January-2026
Download page:	https://github.com/euorrl/HuZhang
Copyright:	Copyright © 2026, Hu Peng, Zhang Zihao – All rights reserved

Contents

Table of Contents	3
List of Figures	5
List of Tables	6
1 Front Page	7
2 Introduction and Scope	8
2.1 System Overview	8
2.2 Scope of the Implementation	8
2.3 Out of Scope	8
2.4 Intended Audience	9
2.5 Acceptance Testing Context	9
3 Implemented Requirements and Functionalities	10
3.1 User Access and Authentication	10
3.2 Home and System Overview	10
3.3 Public Route Exploration	11
3.4 Community Route Sharing	11
3.5 Personal Trip Management	12
3.6 Route Publication Workflow	13
3.7 Geographic Visualization and Mapping	14
3.8 System Health and Backend Monitoring	14
3.9 Summary	14
4 Adopted Development Frameworks and Technologies	15
4.1 Frontend Technologies	15
4.2 Backend Technologies	15
4.3 Database and Persistence	15
4.4 Deployment and Hosting Technologies	15
4.5 Development and Version Control Tools	16
4.6 Rationale for Technology Selection	16
4.7 Summary	16
5 Source Code Structure	17
5.1 Frontend Structure	17
5.2 Backend Structure	18
5.3 Design Rationale	19
6 Testing Strategy and Performed Tests	20
6.1 Testing Environment	20
6.2 Backend Testing	20
6.3 Frontend Testing	20
6.4 Integration Testing	21
6.5 End-to-End and Acceptance Testing	21
6.6 Test Coverage and Limitations	21
6.7 Summary	21
7 Installation and Execution Instructions	22

7.1	System Requirements	22
7.2	Backend Setup and Execution	22
7.2.1	Configuration	22
7.2.2	Execution Procedure	22
7.3	Frontend Setup and Execution	23
7.3.1	Configuration	23
7.3.2	Execution Procedure	23
7.4	Deployed Execution Environment	23
8	Installation and Execution Instructions	24
8.1	Supported Execution Environments	24
8.2	Mobile App and Emulator Availability	24
8.3	Acceptance Testing Capabilities	24
8.4	Requirements to Evaluate the BBP System	25
8.5	Limitations and Mitigation Strategy	25
9	Effort Spent	26
10	Use of AI Tool	27
10.1	Scope of Usage	27
10.2	Human Review and Validation	27
	References	28

List of Figures

1	User Access and Authentication	10
2	Home and System Overview	11
3	Public Route Exploration	11
4	Community Route Sharing	12
5	Personal Trip Management	13
6	Route Publication Workflow	13
7	System Health and Backend Monitoring	14

List of Tables

1	Distribution of effort among group members	26
---	--	----

1 Front Page

This document presents the **Implementation, Integration and Test Deliverable (ITD)** for the **Best Bike Paths (BBP)** project, developed as part of the *Software Engineering 2* course (A.Y. 2025–2026).

The project has been developed by:

Hu Peng	<code>peng.hu@mail.polimi.it</code>
Zhang Zihao	<code>zihao.zhang@mail.polimi.it</code>

The software is implemented as a **web-based prototype**. The complete source code of the project is available at the following GitHub repository:

<https://github.com/euorrl/HuZhang>

The web-based prototype is accessible at the following URL:

<https://hu-zhang.vercel.app>

2 Introduction and Scope

This document describes the implementation details and system characteristics of the Best Bike Paths (BBP) platform. The BBP system is a web-based application designed to support the exploration, recording, and community-based sharing of cycling routes, with a focus on usability, accessibility, and acceptance testing across heterogeneous execution environments.

The purpose of this Implementation and Test Document (ITD) is to provide a structured and technical description of the implemented system architecture, development technologies, source code organization, testing strategy, and available execution infrastructure. The document is intended to support peer review activities and acceptance testing by clearly defining what has been implemented, how it can be deployed, and how it can be evaluated by external groups.

2.1 System Overview

The BBP system provides a unified platform for cyclists to discover public routes, record personal trips, and contribute to a shared community repository of cycling path information.

The system is designed as a browser-accessible web application that integrates a client-side user interface, a backend service layer, and a persistent data storage component. These elements collaborate to provide real-time geographic visualization, trip metadata management, and user-driven route publication.

The platform distinguishes between two primary user roles:

- **Guest users**, who can explore publicly available routes and inspect community-published trip reports.
- **Registered users**, who can additionally record personal trips, manage their trip history, and publish routes to the community repository.

2.2 Scope of the Implementation

The scope of the BBP system implementation includes the following functional and technical boundaries.

The system supports public route discovery through keyword-based search and preference-based filtering, allowing users to prioritize safety or distance when exploring community-published routes.

The system supports personal trip management, including the creation, visualization, and publication of recorded trips. Each trip is associated with geographic track data, distance and duration metrics, safety and surface condition ratings, and optional textual annotations.

The platform provides interactive geographic visualization using digital map services, enabling real-time rendering of routes, automatic zoom and bounding, and visual identification of route endpoints.

The system includes a backend service layer that exposes a RESTful API for authentication, route exploration, trip management, and system health monitoring.

The platform is deployed in a cloud-based environment and is accessible through standard modern web browsers on both desktop and mobile devices, enabling acceptance testing without requiring the installation of native applications.

2.3 Out of Scope

The following functionalities and system characteristics are explicitly considered out of scope for the current prototype and acceptance testing phase.

Native mobile applications for Android or iOS platforms are not implemented. Mobile access is supported exclusively through responsive web interfaces executed in mobile browsers.

The authentication mechanism is implemented as a mock service intended for testing and demonstration purposes. Advanced security features such as password encryption, multi-factor authentication, and role-based authorization policies are not addressed.

Advanced route optimization algorithms, real-time traffic integration, and external navigation service integration are not included in the current implementation.

System scalability, performance benchmarking under high-load conditions, and fault-tolerant deployment strategies are not evaluated within the scope of this document.

2.4 Intended Audience

This document is intended for students and reviewers involved in the peer review and acceptance testing process, as well as academic staff supervising the development of the BBP system.

It assumes a basic understanding of web-based system architectures, RESTful services, and client-server interaction models. The document aims to provide sufficient technical detail to allow independent deployment, execution, and functional validation of the system.

2.5 Acceptance Testing Context

The BBP system is designed to be evaluated in heterogeneous testing environments. Acceptance testing can be performed using any modern web browser on desktop or mobile devices, without the need for platform-specific runtime environments or emulators.

Testers are expected to validate functional requirements such as public route exploration, trip recording, route publication, and map-based visualization, as well as system accessibility and backend availability through the exposed health monitoring interface.

This design choice ensures that peer reviewers can execute and evaluate the system regardless of their underlying operating system or hardware configuration.

3 Implemented Requirements and Functionalities

This section describes the functional requirements that have been implemented in the Best Bike Paths (BBP) system prototype and the corresponding system functionalities made available to end users. The system is designed as a web-based platform that supports public route exploration, community-driven trip sharing, and personal trip management through a unified browser-accessible interface.

3.1 User Access and Authentication

The system provides a lightweight user access mechanism intended for prototyping and acceptance testing purposes.

Users can access the platform as either guests or registered users. Guest users are allowed to explore public routes and view community-published trip reports without authentication. Registered users are provided with additional functionalities, including trip recording and route publication.

A mock authentication mechanism is implemented to simulate user registration and login. When a user submits credentials that do not exist in the system, a new user profile is automatically created and stored in the database. This approach allows acceptance testers to easily generate test accounts without requiring administrative intervention.



Figure 1: User Access and Authentication

3.2 Home and System Overview

The home interface serves as the main entry point to the BBP system. It presents a general description of the platform objectives and provides direct navigation to the primary system modules.

The interface highlights the distinction between public and registered user functionalities and offers quick access to route exploration and community features. This design supports intuitive onboarding for first-time users and reviewers.

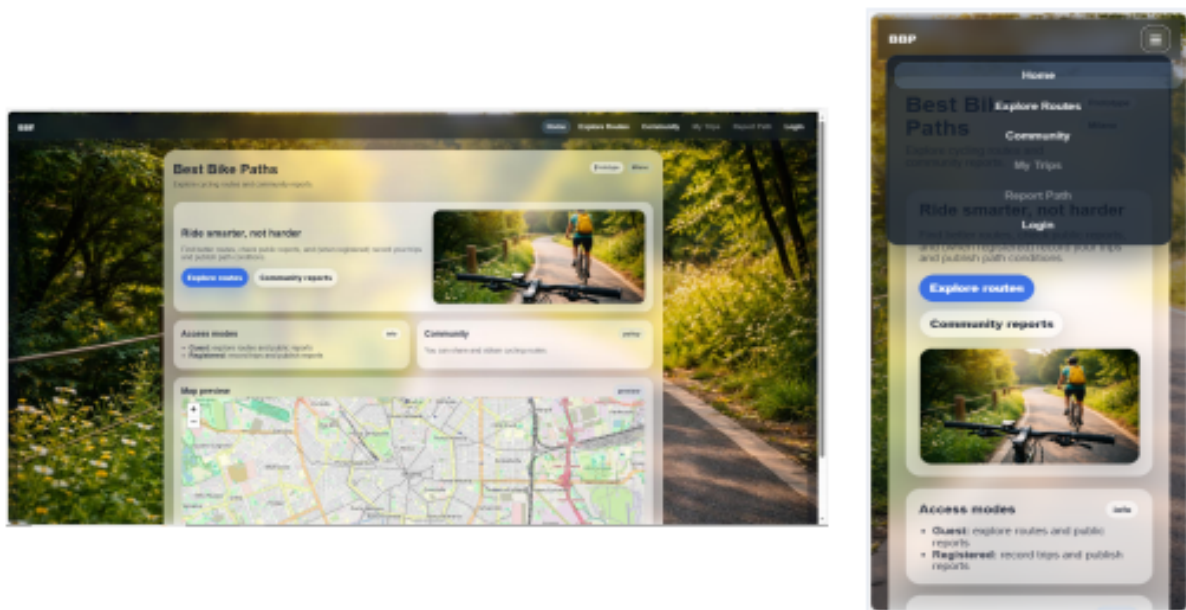


Figure 2: Home and System Overview

3.3 Public Route Exploration

The system implements a public route exploration module that allows users to search for published routes based on textual origin and destination keywords.

Users can specify route preferences, such as prioritizing safety or minimizing distance. The backend processes these parameters and returns a filtered set of matching public trips.

Search results are displayed in a structured list format, including metadata such as route distance, safety rating, and surface condition rating. Users can select a specific result to visualize the corresponding route on an interactive map.

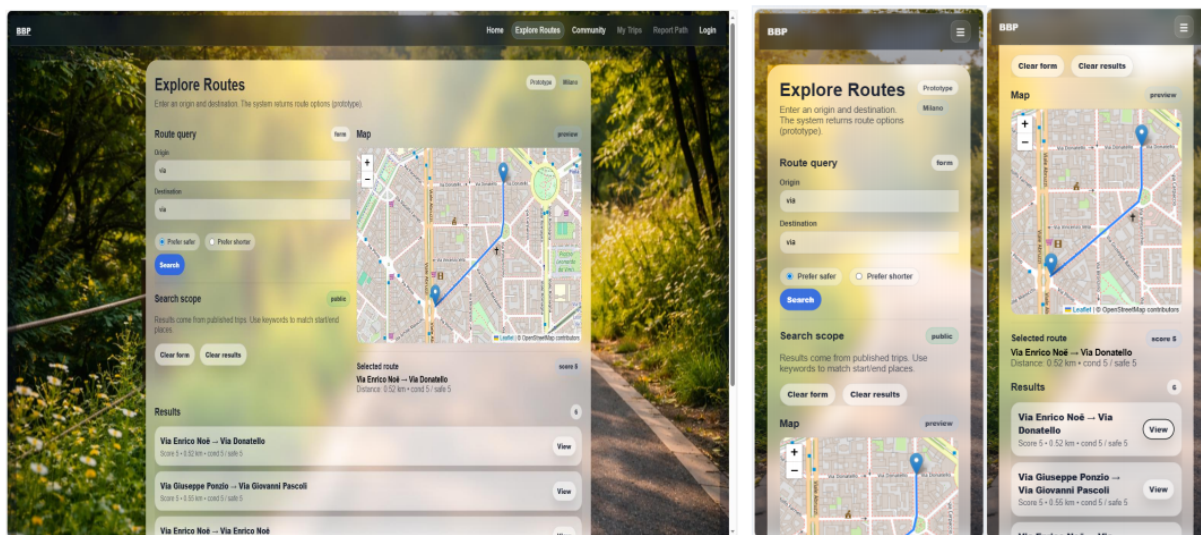


Figure 3: Public Route Exploration

3.4 Community Route Sharing

The community module enables users to browse and inspect routes published by other registered users.

The system retrieves all trips marked as publicly available and presents them in a searchable and filterable list. Users can perform keyword-based filtering based on route endpoints and associated notes.

For each selected community report, the system displays detailed trip information and renders the recorded geographic track on an interactive map. Start and end points are visually highlighted using map markers, and the full path is shown as a polyline.



Figure 4: Community Route Sharing

3.5 Personal Trip Management

Registered users can access a personal trip management module that allows them to record, review, and manage their own cycling trips.

Each trip includes metadata such as date, total distance, duration, surface condition rating, and safety rating. Users can also attach optional textual notes describing the trip conditions.

The system maintains a historical list of all trips associated with the current user account and supports detailed inspection of individual trips through map-based visualization.

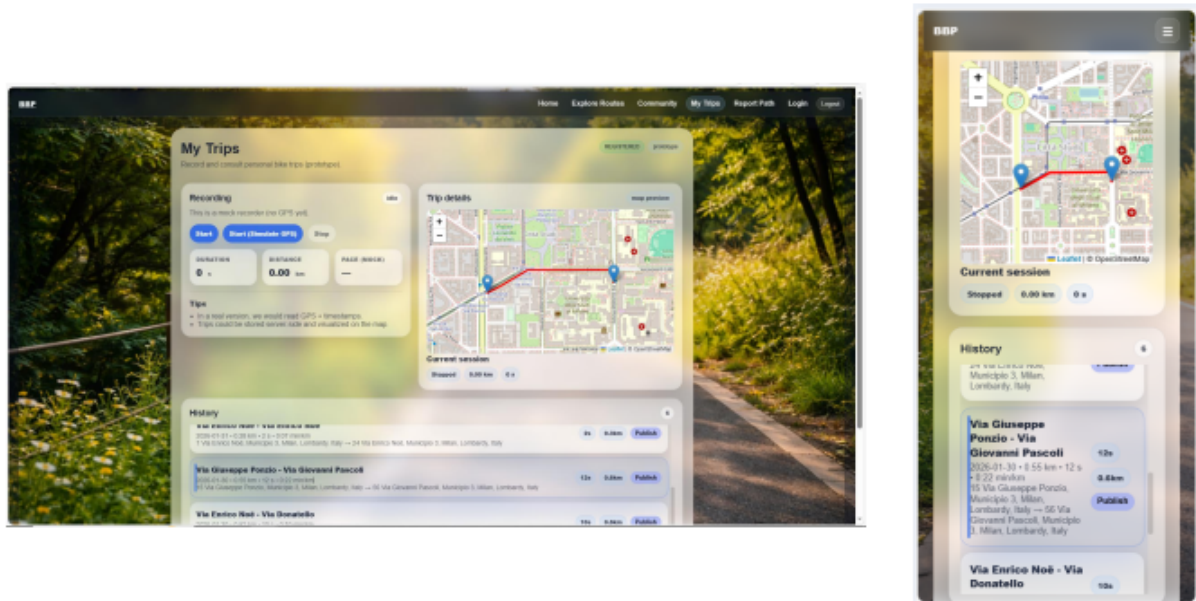


Figure 5: Personal Trip Management

3.6 Route Publication Workflow

The system implements a publication workflow that enables users to convert personal trips into publicly visible routes.

Users can select one of their recorded trips and publish it to the community repository. During this process, the system marks the trip as publicly accessible and makes it available for search and inspection by other users through the public route exploration and community modules.

This functionality supports collaborative data generation and community-driven improvement of cycling route information.



Figure 6: Route Publication Workflow

3.7 Geographic Visualization and Mapping

The system integrates an interactive geographic visualization component based on open map services.

All route-related modules support real-time rendering of trip tracks on a digital map. The system visualizes recorded GPS tracks as polylines and highlights the starting and ending locations with distinct markers.

Map views automatically adjust their zoom and bounds to fit the selected route, ensuring consistent visibility across different screen sizes and devices.

3.8 System Health and Backend Monitoring

The backend exposes a dedicated health endpoint that allows verification of service availability and deployment status.

This functionality supports acceptance testing and system validation by enabling reviewers to confirm that the backend service is operational and reachable before executing functional test cases.

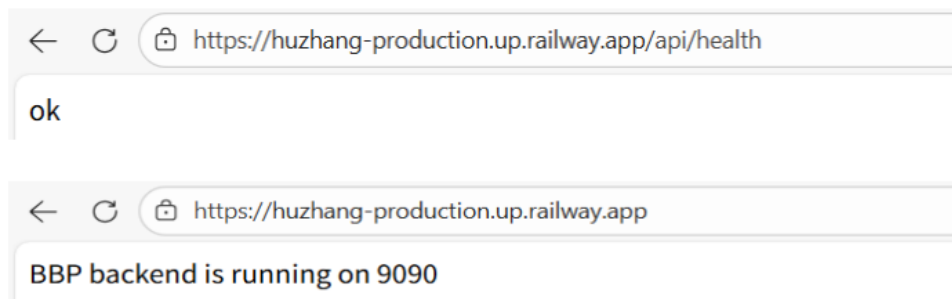


Figure 7: System Health and Backend Monitoring

3.9 Summary

The implemented functionalities of the BBP system cover the full lifecycle of route discovery, trip recording, and community-based route sharing. The system supports both guest and registered user workflows and provides a consistent, browser-based user experience for public exploration, personal trip management, and collaborative route publication.

The modular implementation of these features enables independent testing of each functional area and supports structured acceptance testing by peer reviewers across heterogeneous execution environments.

4 Adopted Development Frameworks and Technologies

The Best Bike Paths (BBP) system is implemented as a web-based, distributed application composed of a frontend client, a backend service layer, and a managed cloud database. The adopted frameworks and technologies were selected to ensure modularity, scalability, ease of deployment, and compatibility with modern web browsers and development tools.

The overall architecture follows a client-server model, where the frontend communicates with the backend exclusively through RESTful APIs. This separation of concerns allows independent development, testing, and deployment of each subsystem.

4.1 Frontend Technologies

The frontend of the BBP system is implemented as a Single Page Application (SPA) using the Vue.js framework. Vue.js was selected for its component-based architecture, lightweight runtime, and strong ecosystem support for reactive user interfaces.

The development environment is based on the Vite build tool, which provides fast development server startup, efficient hot module replacement, and optimized production builds. This enables rapid prototyping and efficient iteration during development.

Routing and navigation between views are managed using a client-side routing library, enabling seamless transitions between application pages without full page reloads. State related to user authentication and session management is handled through a centralized client-side store.

For geographic visualization and interactive route rendering, the system integrates the Leaflet library together with OpenStreetMap tile services. This combination enables real-time map rendering, route polyline visualization, and marker-based representation of trip start and end points.

4.2 Backend Technologies

The backend subsystem is implemented using the Spring Boot framework and is developed in the Java programming language. Spring Boot provides a robust foundation for building RESTful web services, including dependency injection, configuration management, and embedded server support.

The backend exposes a set of REST endpoints responsible for authentication, trip management, route exploration, health monitoring, and GPS data handling. These endpoints serve as the primary interface between the frontend client and the persistence layer.

Data transfer between frontend and backend is performed using JSON-based Data Transfer Objects (DTOs), ensuring a consistent and platform-independent communication format. Object-relational mapping and structured data handling are supported through mapper components that translate between internal domain models and external representations.

4.3 Database and Persistence

The system uses a managed cloud-based relational database service for persistent data storage. This service provides structured storage for users, trips, route metadata, and associated geographic track data.

Database connectivity and configuration are managed through externalized environment variables and configuration files, allowing seamless adaptation between local development and deployed environments. This approach improves security by avoiding hard-coded credentials and simplifies deployment across different platforms.

4.4 Deployment and Hosting Technologies

The frontend is deployed on a cloud-based static hosting platform optimized for modern web applications. This platform provides automatic build pipelines, global content delivery, and HTTPS support, enabling reliable and secure access to the user interface from standard web browsers.

The backend is deployed as a cloud-hosted service using a platform that supports containerized or managed Java applications. This environment enables scalable deployment, remote monitoring, and consistent service availability.

Cross-origin communication between the frontend and backend is enabled through explicit configuration of Cross-Origin Resource Sharing (CORS) policies at the backend level, ensuring secure and controlled access to the exposed REST endpoints.

4.5 Development and Version Control Tools

Source code management is handled using a distributed version control system, enabling collaborative development, change tracking, and rollback capabilities. The project repository is structured to clearly separate frontend, backend, and documentation artifacts.

The development workflow is supported by modern integrated development environments and browser-based developer tools, which are used for debugging, network inspection, and performance analysis during both development and acceptance testing phases.

4.6 Rationale for Technology Selection

The selected technology stack emphasizes portability, modularity, and ease of peer evaluation. By adopting a web-based architecture, the system can be executed and tested on any modern device equipped with a standard web browser, without requiring specialized operating systems, mobile emulators, or native application installation.

This design choice directly supports the acceptance testing process by ensuring that reviewers can easily access and validate system functionality using commonly available hardware and software resources.

4.7 Summary

The adopted development frameworks and technologies provide a cohesive and extensible foundation for the BBP system. The combination of a reactive frontend, a robust backend service layer, and a managed cloud database ensures reliable system behavior, efficient development workflows, and straightforward deployment and acceptance testing across heterogeneous environments.

5 Source Code Structure

The Best Bike Paths (BBP) system follows a modular, layered source code organization that separates presentation logic, application logic, and data access responsibilities. The repository is structured into two main subsystems: a web-based frontend implemented using the Vue.js framework and a backend service implemented using Spring Boot and Java 8. This separation supports independent development, deployment, and testing of each subsystem while maintaining a clear and well-defined interface between them.

5.1 Frontend Structure

The frontend source code is located under the `src/` directory and follows a component-based architecture consistent with modern single-page application (SPA) design principles. The structure is organized by functional responsibility rather than by technical role, improving maintainability and scalability.

API Layer

The `src/api/` directory contains JavaScript modules responsible for communication with the backend REST services. These modules abstract all HTTP interactions and provide a clean interface for the view layer. The main files include:

- `auth.js` – Handles user authentication and mock login or registration requests.
- `trips.js` – Manages retrieval and publication of trip and route data, including public community reports.
- `gps.js` – Provides support for geolocation and track-related requests.
- `health.js` – Verifies backend service availability through health-check endpoints.
- `http.js` – Defines a centralized HTTP client configuration, including base URL and request handling.

This separation ensures that changes to backend endpoints or network configurations can be handled without affecting the user interface logic.

Pages and Components

The `src/pages/` directory contains the main view-level components, each representing a major functional area of the system:

- `Home.vue` – Provides the system entry point and overview.
- `ExploreRoutes.vue` – Supports public route search, filtering, and map-based visualization.
- `Community.vue` – Displays community-published routes and detailed trip visualization.
- `MyTrips.vue` – Allows registered users to manage and review their personal trips.
- `ReportPath.vue` – Enables users to publish recorded trips as public community routes.
- `Login.vue` – Implements the mock authentication interface.

Reusable UI elements and layout logic are placed in `src/components/` and `src/layouts/`, where `MainLayout.vue` defines the global navigation structure and shared visual elements such as the top navigation bar.

Routing and State Management

The `src/router/index.js` file defines all client-side routes and maps URLs to their corresponding page components, enabling seamless navigation within the single-page application. Session state and authentication status are managed centrally in the `src/store/` directory, ensuring consistent access control across protected views such as *My Trips* and *Report Path*.

Assets and Configuration

Static resources such as images and global styles are stored in the `src/image/` and `src/assets/` directories. Build and deployment configuration is handled through `vite.config.js`, `vercel.json`, and standard Node.js package descriptors, supporting automated frontend deployment on cloud platforms.

5.2 Backend Structure

The backend subsystem is implemented using Spring Boot and follows a layered architecture that separates configuration, request handling, business logic, and data persistence.

Configuration Layer

The `config` package contains infrastructure-related components. In particular, `CorsConfig` defines cross-origin request policies, enabling secure communication between the deployed frontend and backend services across different domains.

Application-level configuration, including database connectivity and environment-specific parameters, is defined in `application.yml`.

Controller Layer

The `controller` package contains REST controllers that expose the system's public API. Each controller is responsible for handling HTTP requests, validating input parameters, and delegating processing to the appropriate service or data access layer. Key controllers include:

- `AuthController` – Handles user login and mock registration functionality.
- `ExploreRouteController` – Provides public route search and filtering endpoints.
- `TripController` – Manages personal and public trip-related operations.
- `GpsController` – Supports GPS and track data submission and retrieval.
- `HealthController` – Exposes service health-check endpoints.

This design ensures that the API layer remains thin and focused on request-response handling rather than business logic.

Data Transfer and Persistence

The `dto` package contains data transfer objects such as `ExploreRoutePublicTripDto`, which define the structure of data exchanged between backend and frontend, ensuring a stable and well-defined API contract.

Database access is handled through the `mapper` package, which uses MyBatis mappers such as `TripMapper`, `UserMapper`, and `ExploreRouteMapper`. These components encapsulate all SQL operations and isolate persistence logic from the controller layer.

Application Entry Point

The `BbpBackendApplication` class represents the main entry point of the backend system and is responsible for bootstrapping the Spring Boot application context and initializing all configured components.

5.3 Design Rationale

The adopted source code structure emphasizes separation of concerns, modularity, and scalability. The frontend and backend can be developed, deployed, and tested independently, while internal layering within each subsystem reduces coupling and improves maintainability. This organization supports future extension of system functionality, such as introducing native mobile clients or additional data services, without requiring major architectural changes.

6 Testing Strategy and Performed Tests

The testing strategy adopted for the Best Bike Paths (BBP) system is designed to ensure functional correctness, reliability, and usability across both frontend and backend subsystems. Given the distributed and web-based nature of the application, testing activities focus on validating the correctness of REST APIs, the consistency of data exchange between components, and the usability of the system through real user interaction scenarios.

The testing process follows a multi-layered approach that includes unit-level verification of backend services, integration testing between frontend and backend, and end-to-end acceptance testing through browser-based execution.

6.1 Testing Environment

The system is tested in both local and deployed environments to ensure consistent behavior across development and production settings. The main testing environments include:

- **Local environment:** Frontend running on a local development server and backend running as a Spring Boot service. This setup is used for rapid debugging and functional validation during development.
- **Deployed environment:** Frontend deployed on a cloud hosting platform and backend deployed as a remote service. The database is hosted on a managed cloud database service. This environment is used to validate real-world deployment behavior, cross-origin communication, and system stability under realistic network conditions.

All acceptance and integration tests are executed using modern web browsers on desktop and mobile devices, ensuring compatibility across different platforms without requiring native application runtimes.

6.2 Backend Testing

Backend testing focuses on verifying the correctness and robustness of the RESTful API layer. Each controller endpoint is tested using direct HTTP requests to validate:

- Correct request parameter handling and validation
- Proper HTTP status code responses
- Consistent response data structure
- Successful interaction with the persistence layer

A dedicated health-check endpoint is used to confirm service availability before performing functional tests. Database connectivity is verified through test endpoints that execute controlled read and write operations to ensure that the backend can reliably access the remote database service.

6.3 Frontend Testing

Frontend testing is primarily conducted through manual and scenario-based testing in a web browser, focusing on user interaction flows and visual feedback. The following aspects are validated:

- Correct rendering of all main views and navigation elements
- Input validation for route search, login, and trip reporting forms
- Dynamic data loading from backend services

- Error handling and user feedback for network and service failures

Special attention is given to the integration of map visualization features, ensuring that geographic data, route tracks, and markers are displayed consistently across different devices and screen sizes.

6.4 Integration Testing

Integration testing validates the communication between frontend and backend components through the deployed REST API. The tests focus on:

- Cross-origin request handling between frontend and backend services
- Data consistency between database records and frontend visualizations
- Correct mapping of backend data transfer objects to frontend view models

These tests are performed by executing realistic user workflows, such as publishing a trip, retrieving it through the community interface, and visualizing the corresponding route on the interactive map.

6.5 End-to-End and Acceptance Testing

End-to-end testing is conducted using full user scenarios that simulate real-world usage of the system. The main acceptance test cases include:

- User login or mock registration
- Recording and publishing a trip
- Searching for public routes through the *Explore Routes* interface
- Browsing and selecting community-published trips
- Visualizing trip details and route tracks on the map

These tests are executed on both desktop and mobile web browsers to ensure usability and functional consistency across platforms. This approach supports peer review and acceptance testing without requiring specialized mobile development environments or native application installation.

6.6 Test Coverage and Limitations

The performed tests cover the complete functional workflow of the system, from data creation to public visualization. However, automated unit testing frameworks and performance or load testing tools are not fully integrated into the current prototype. As a result, non-functional properties such as system scalability and stress behavior are evaluated only through limited manual testing.

Future work may include the introduction of automated test pipelines and continuous integration support to improve regression testing and long-term system maintainability.

6.7 Summary

The adopted testing strategy ensures that the BBP system is functionally correct, interoperable across components, and usable in realistic deployment conditions. By combining backend validation, frontend scenario testing, and full end-to-end workflows, the system demonstrates a consistent and reliable behavior suitable for acceptance testing and peer evaluation.

7 Installation and Execution Instructions

This section describes the required software, configuration steps, and execution procedure needed to install and run the Best Bike Paths (BBP) system.

The system follows a client-server architecture composed of a web-based frontend, a backend REST service, and a cloud-hosted relational database.

7.1 System Requirements

To install and execute the BBP system, the following resources are required:

- A modern web browser with JavaScript enabled (e.g., Google Chrome, Microsoft Edge, Safari)
- An active Internet connection
- A development environment supporting Java 8 and Apache Maven
- Node.js (version 18 or later) and npm for frontend development

7.2 Backend Setup and Execution

The backend component is implemented as a Java 8 Spring Boot application using Maven as the build system. It exposes a RESTful API that handles user operations, trip management, and public route queries.

7.2.1 Configuration

The backend requires access to a PostgreSQL-compatible database hosted on Supabase. The database connection parameters are configured through environment variables, which must be defined prior to execution.

The following parameters must be provided:

- SUPABASE_URL
- SUPABASE_ANON_KEY
- DB_HOST
- DB_PORT
- DB_NAME
- DB_USER
- DB_PASSWORD

7.2.2 Execution Procedure

To start the backend locally, the following commands can be executed:

```
cd src/backend/springboot
mvn spring-boot:run
```

Once started, the backend service is available by default at:

`https://huzhang-production.up.railway.app`

A basic health check endpoint is provided at:

`/api/health`

7.3 Frontend Setup and Execution

The frontend is implemented as a web application using Vue 3 and the Vite build system. It provides the user interface for route exploration, trip visualization, and community interaction.

7.3.1 Configuration

The frontend communicates with the backend through a configurable API base URL, which is defined via an environment variable:

- `VITE_API_BASE_URL`

This variable must point to the backend service endpoint (e.g., `https://huzhang-production.up.railway.app` for local execution or the deployed backend URL for production usage).

7.3.2 Execution Procedure

To install dependencies and run the frontend locally, the following commands can be used:

```
cd src/frontend
npm install
npm run dev
```

The web application will be accessible at:

`https://hu-zhang.vercel.app`

7.4 Deployed Execution Environment

The BBP system is also available as a fully deployed, remotely accessible platform:

- Frontend deployed on Vercel
- Backend deployed on Railway
- Database hosted on Supabase

This deployment allows the system to be executed and evaluated without requiring local installation.

8 Installation and Execution Instructions

This section reports the execution and testing resources available to our group during the acceptance testing and peer review phases. The goal is to ensure that, if we are assigned another group's artifact for evaluation, we can realistically execute it and assess its main functionalities.

8.1 Supported Execution Environments

Our group can reliably execute and test **web-based applications** (client-server systems accessible through a browser) using the following environments:

- **Desktop browsers:** Google Chrome / Chromium-based browsers and Mozilla Firefox on Windows.
- **Mobile browsers:** Chrome / system browser on Android devices.
- **Network/API testing:** direct interaction with REST endpoints using browser developer tools and HTTP clients (e.g., Postman or curl).

Therefore, we can perform acceptance testing for systems delivered as:

- A deployed web application reachable via a public URL; and/or
- A locally runnable web application, provided that installation instructions and configuration parameters are given.

8.2 Mobile App and Emulator Availability

With respect to mobile execution, our available infrastructure is as follows:

- **Android (physical device):** available. We can execute and test applications on Android devices.
- **Android/iOS emulator:** available on Windows via Android Studio Emulator.

As a consequence, we cannot ensure the capability to execute or fully validate **native iOS applications** that require Xcode/iOS Simulator, unless the application is additionally delivered in a web-accessible form.

8.3 Acceptance Testing Capabilities

Given the above infrastructure, we can perform the following acceptance testing activities:

- **End-to-end functional testing** of user-facing features through the UI (navigation, forms, data visualization, and map interaction).
- **Cross-device verification** by repeating key scenarios on both desktop and mobile browsers.
- **API-level checks** to confirm endpoint reachability, authentication/login flows, and payload correctness.
- **Non-functional checks (basic):** responsiveness on mobile screens, basic performance observations (page load and API latency), and error handling verification.

8.4 Requirements to Evaluate the BBP System

The BBP system is delivered as a web-based platform and can be tested without native mobile runtimes. The only resources required to execute and evaluate it are:

- A modern web browser with JavaScript enabled (desktop or mobile)
- Internet connectivity to reach the deployed services (frontend and backend)

Optionally, for deeper inspection of integration aspects, the following tools may be used:

- Browser developer tools (Network tab) for request/response inspection
- HTTP client tools (e.g., Postman or curl) to call REST endpoints directly

8.5 Limitations and Mitigation Strategy

The main limitation of our acceptance testing infrastructure is the lack of guaranteed access to iOS execution (device or macOS-based simulator). To mitigate this, if we receive for peer review a system that includes iOS-specific deliverables, we will:

- request a **web-accessible demo** (if available), or
- evaluate the **backend/API and documentation** as supporting evidence, and
- execute the **Android build and/or emulator version** when provided as an alternative.

This ensures that we can still provide meaningful feedback and verification for the majority of the delivered functionality, even when an iOS-only runtime is not executable in our environment.

9 Effort Spent

The following table displays the number of hours each group member spent on the various sections of the document. Please note that this division is only approximate, and each section still required the collaboration of all members.

Table 1: Distribution of effort among group members

Member	Chapter 1	Chapter 2	Chapter 3	Chapter 4	Chapter 5	Chapter 6	Chapter 7	Chapter 8
Hu Peng	1	2	4	3	2	3	2	1
Zhang Zihao	1	3	3	4	3	2	3	2

10 Use of AI Tool

During the development of the Best Bike Paths (BBP) system, AI-based tools were used as a supporting resource to enhance productivity, improve code quality, and refine documentation. The use of these tools was strictly limited to assistance purposes and did not replace the design decisions, implementation work, or validation activities performed by the development team.

10.1 Scope of Usage

AI tools were primarily employed in the following areas:

- **Debugging Support:** Analyzing error messages and unexpected runtime behavior, such as API response format mismatches and deployment-related configuration issues, to help identify potential causes and corrective actions.
- **User Interface Refinement:** Providing suggestions for layout consistency and responsive design improvements for mobile and desktop web interfaces.

10.2 Human Review and Validation

All system functionalities and documentation content were manually reviewed and validated by the project members. System behavior was evaluated through practical execution and testing, including front-end and back-end interactions, database read and write operations, and consistency checks across the main user interaction scenarios. Identified issues were analyzed and resolved through debugging tools, log inspection, and iterative testing, ensuring stability and reliability in the deployed environment.

References